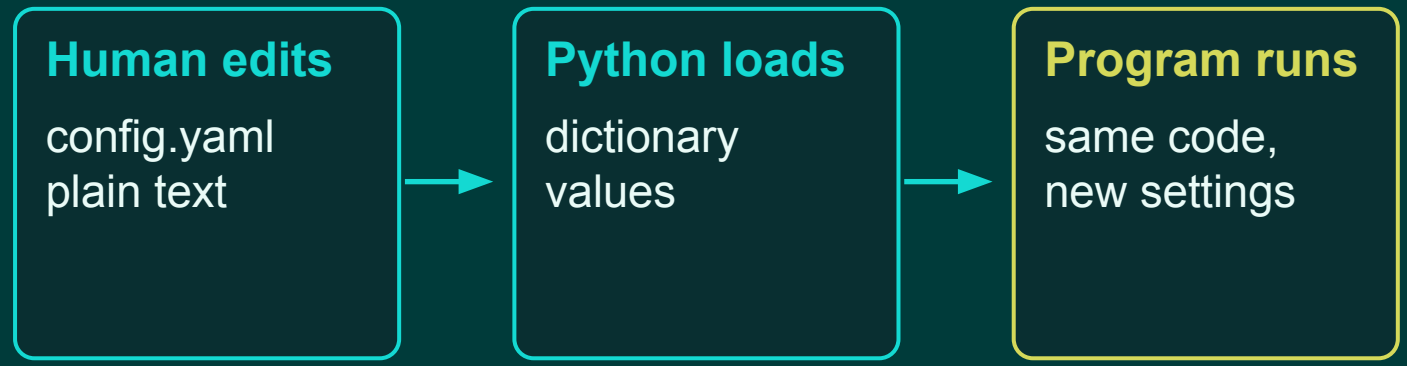
APPENDIX A

YAML for Configuration Files

A one-block appendix lesson: readable settings outside your Python code

Why this appendix exists

- YAML often appears in config files
- It looks simple, but indentation matters
- Python can read it with a third-party package
- Goal: understand enough to use it safely

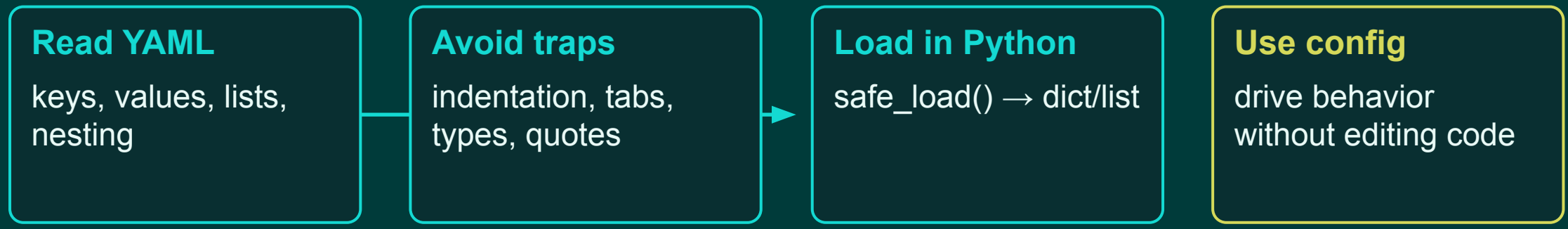


Mental model: YAML is data, not Python code.

APPENDIX A

Block Plan

30 minutes instruction + 30 minutes guided practice



By the end: students can convert a JSON-style config to YAML and load it into a Python program.

APPENDIX A

Why Put Settings in YAML?

Configuration changes more often than program logic

Hard-coded setting

maximum_errors = 5
inside the Python file

→

Configuration file

maximum_errors: 5
outside the Python file

→

Same program

read settings at startup
then process records

Good configuration keeps values easy to change without scattering magic numbers through the code.

```
allowed_statuses:
  - active
  - inactive
maximum_errors: 5
output_file: data/output/results.csv
```

APPENDIX A

YAML Basics: Keys and Values

Most configuration starts with name: value pairs

```
source_system: maintenance
processing_rate: 250
maximum_errors: 5
output_enabled: true
```

Key

The name on the left side.
Examples: source_system, processing_rate

Value

The data on the right side.
Examples: maintenance, 250, true

Read source_system: maintenance as “the source_system setting has the value maintenance.”

APPENDIX A

Scalar Values

Strings, numbers, booleans, and null-like values

```
project_name: record_processor
record_limit: 1000
error_rate: 0.05
enabled: true
owner: null
```

Common conversions

Values may become Python types after loading.

```
"record_processor"  # str
1000                # int
0.05                # float
True                # bool
None                # none-like
```

Important: a value that looks like a number or boolean may stop being a string.

APPENDIX A

Lists Use Dashes

A YAML sequence becomes a Python list

```
allowed_sources:
  - maintenance
  - logistics
  - finance

supported_extensions:
  - .csv
  - .json
```

```
config["allowed_sources"]

# [
#   "maintenance",
#   "logistics",
#   "finance",
# ]
```

The dash means “another item in this list.” Items at the same indentation level belong together.

APPENDIX A

Nesting Depends on Indentation

Indented values belong to the key above them

```yaml
limits:
  maximum_errors: 5
  maximum_file_size_mb: 500

output:
  clean_file: data/output/clean.csv
  rejected_file: data/output/rejected.csv
```

```
maximum_errors = (
    config["limits"]["maximum_errors"]
)

clean_file = (
    config["output"]["clean_file"]
)
```

Mental model: indentation creates the tree structure.

APPENDIX A

YAML, JSON, and Python Dictionaries

They can represent similar information with different syntax

YAML

Human-friendly config
Indentation-sensitive
No commas required

```
maximum_errors: 5
enabled: true
```

JSON

Strict data format
Great for APIs
Uses braces and commas

```
{
  "maximum_errors": 5,
  "enabled": true
}
```

Python dict

Python source code
Uses True/False/None
Not YAML text

```
{
    "maximum_errors": 5,
    "enabled": True,
}
```

Do not paste YAML directly into Python and expect it to run.

APPENDIX A

Loading YAML in Python

YAML support usually comes from a third-party package

Install once

```
python -m pip install pyyaml
```

```
from pathlib import Path
import yaml

config_path = Path("config/settings.yaml")

with open(config_path, encoding="utf-8") as f:
    config = yaml.safe_load(f)

print(config["maximum_errors"])
```

Why safe_load?

Use safe_load() for normal config files. It loads ordinary data without enabling arbitrary Python object construction.

Result

config is now a normal Python dictionary or list.

APPENDIX A

Common YAML Mistakes

Small formatting choices can change the data structure

```
allowed_statuses:
  - active
  - inactive
```

Good: list items line up

```
allowed_statuses:
  - active
    - inactive
```

Bad: indentation changed meaning

```
project_id: 00123
project_id_text: "00123"
path: "data/output/results.csv"
```

Quote strings when leading zeros, punctuation, or exact text matter.

Instructor emphasis

Use spaces, not tabs. Keep indentation consistent. Validate loaded values before relying on them.

APPENDIX A

Guided Exercise 1: Read a YAML Config

Create config/settings.yaml and load it into Python

```
allowed_sources:
  - maintenance
  - logistics
  - finance

limits:
  maximum_errors: 5
  maximum_record_count: 20000
```

Task

1. Save this YAML file.
2. Load it with yaml.safe_load().
3. Print allowed_sources.
4. Print maximum_errors.

Timing: 10 minutes

Check

Are allowed_sources a list?
Is maximum_errors an integer?

APPENDIX A

Guided Exercise 2: Use Config in Validation

Replace hard-coded values with config-driven values

```
def is_allowed_source(source, allowed):
    cleaned = source.strip().lower()

    return cleaned in allowed

allowed = config["allowed_sources"]

print(is_allowed_source("LOGISTICS", allowed))
```

Task

1. Load allowed_sources from YAML.
2. Pass it into the function.
3. Test active values and rejected values.

Timing: 10 minutes

Design habit: functions receive config values instead of secretly reading global variables.

APPENDIX A

Applied Challenge: Convert JSON Config to YAML

Use YAML to drive a small record-processing rule set

```
{
  "allowed_statuses": ["active", "inactive"],
  "maximum_errors": 5,
  "warning_error_threshold": 1
}
```

```
allowed_statuses:
  - active
  - inactive
maximum_errors: 5
warning_error_threshold: 1
```

Challenge steps

1. Convert the JSON to YAML.
2. Load it in Python.
3. Validate a record status.
4. Classify error_count as ready, warning, or rejected.

Timing: 25 minutes

Appendix A Wrap-Up

YAML is small, practical, and easy to misuse without indentation discipline

Know the syntax

key: value
lists use dashes
nesting uses indentation

Load safely

Use yaml.safe_load()
Treat result as dict/list
Validate assumptions

Use it well

Put settings in config
Pass values into functions
Keep code reusable

Knowledge check: What Python type do you expect from a YAML mapping? What about a YAML sequence?

Takeaway: YAML is not magic. It is structured text that becomes normal Python data after loading.